

Name: Faizan Khurram

10 Pearls Project Report
(AQI Predictor)

Email: faizan.khurram@hotmail.com

Github Repo Link:
<https://github.com/faizankhurram/AQI-Predictor>

1. Problem Statement

Karachi faces recurring poor air quality from traffic, industry, and seasonal dust. Residents need more than a single current reading i.e they need a short-horizon outlook to plan outdoor activity and early response. This project delivers an automated 3-day US AQI forecast in which forecasts are produced by iterative one-hour-ahead prediction, not by training separate +24h/+48h/+72h models (+24h, +48h, +72h) for Karachi using hourly Open-Meteo data, a cloud feature store, daily model retraining, and a public Streamlit dashboard.

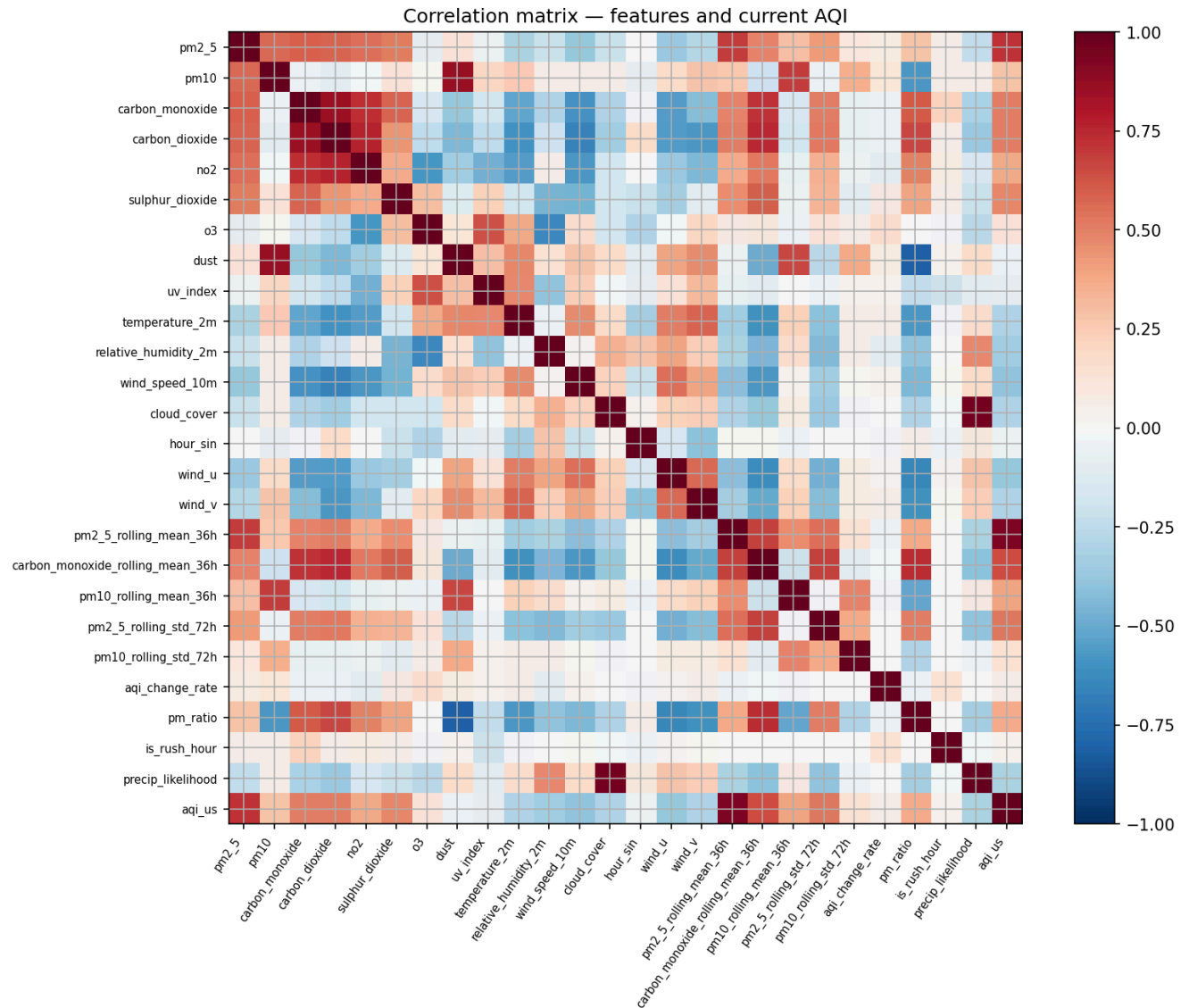
2. Preprocessing and Feature Engineering:

- Performed Short forward-fill (limit 2) -> linear interpolation -> backward-fill on numeric columns.
- Performed IQR capping at $1.5 \times \text{IQR}$ per numeric column (skip bounded/circular columns, humidity, cloud cover, wind direction).
- Full pollutant panel from Open-Meteo i.e PM2.5, PM10, CO, CO₂, NO₂, SO₂, O₃, dust, UV index plus temperature, humidity, wind speed/direction and cloud cover.
- Cyclic **hour_sin**, wind U/V components from speed + direction, 36h rolling means (PM2.5, PM10, CO), 72h rolling stds (PM2.5, PM10). AQI change rate, PM2.5/PM10 ratio, rush-hour flag, precipitation-likelihood (humidity $\times$ cloud cover).
- Single next-hour US AQI (**aqi_us.shift(-1)**). *aqi_us* is not used as a model input.
- Time-ordered 80/20 train/test (not a fixed-day holdout).
- MinMaxScaler at training time.
- Multi-day dashboard horizons (+24h / +48h / +72h) are derived at inference by recursive one-hour-ahead forecasting (96 steps), not by separate training targets.

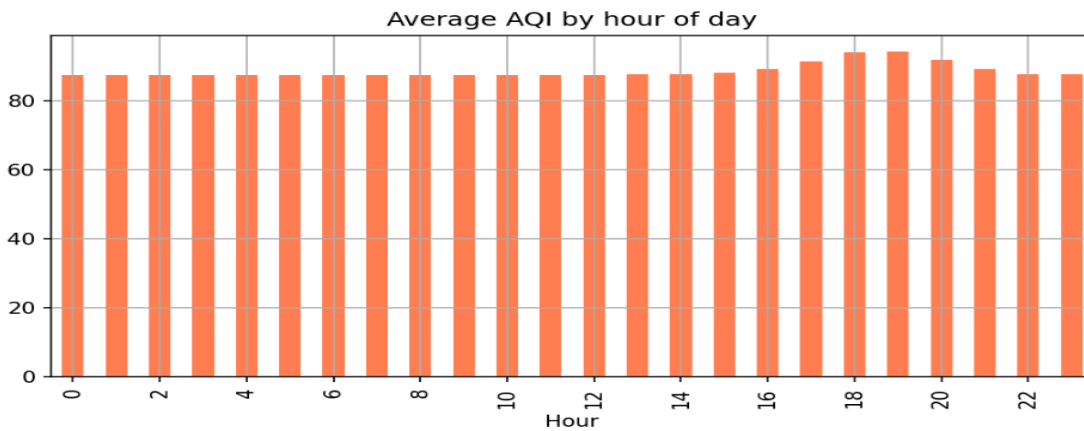
3. Exploratory Data Analysis Findings:

The following are some of the important highlights and visuals from the performed EDA:

(i): PM2.5 dominates US AQI (correlation >0.95).

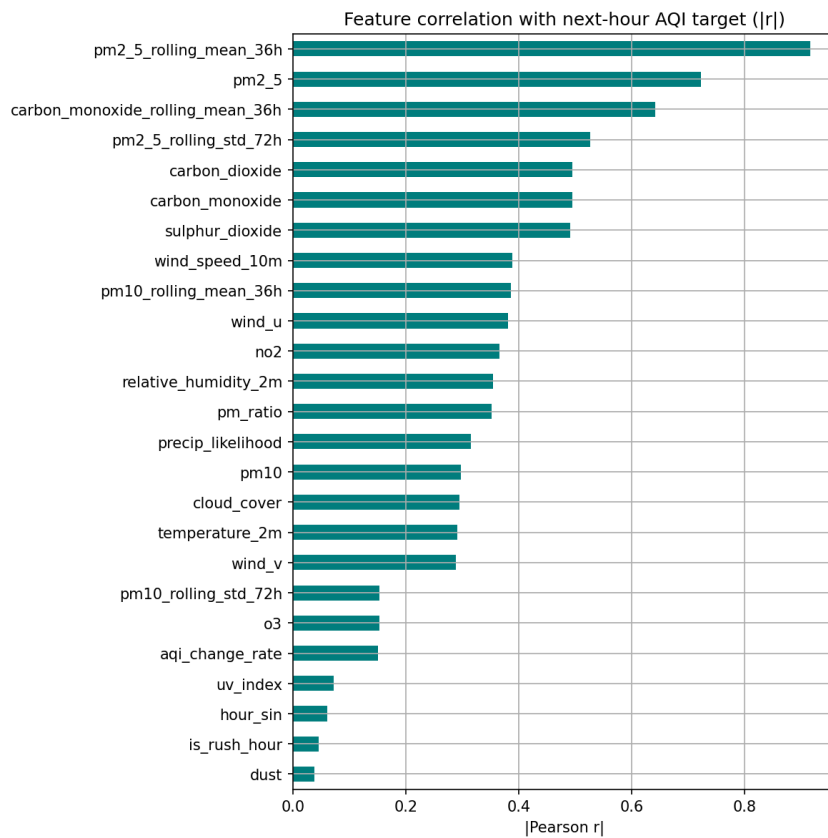


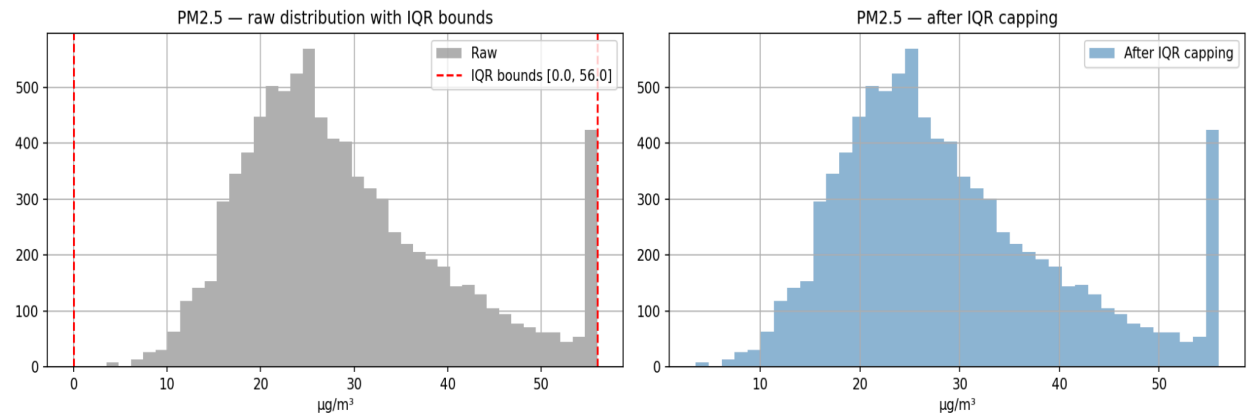
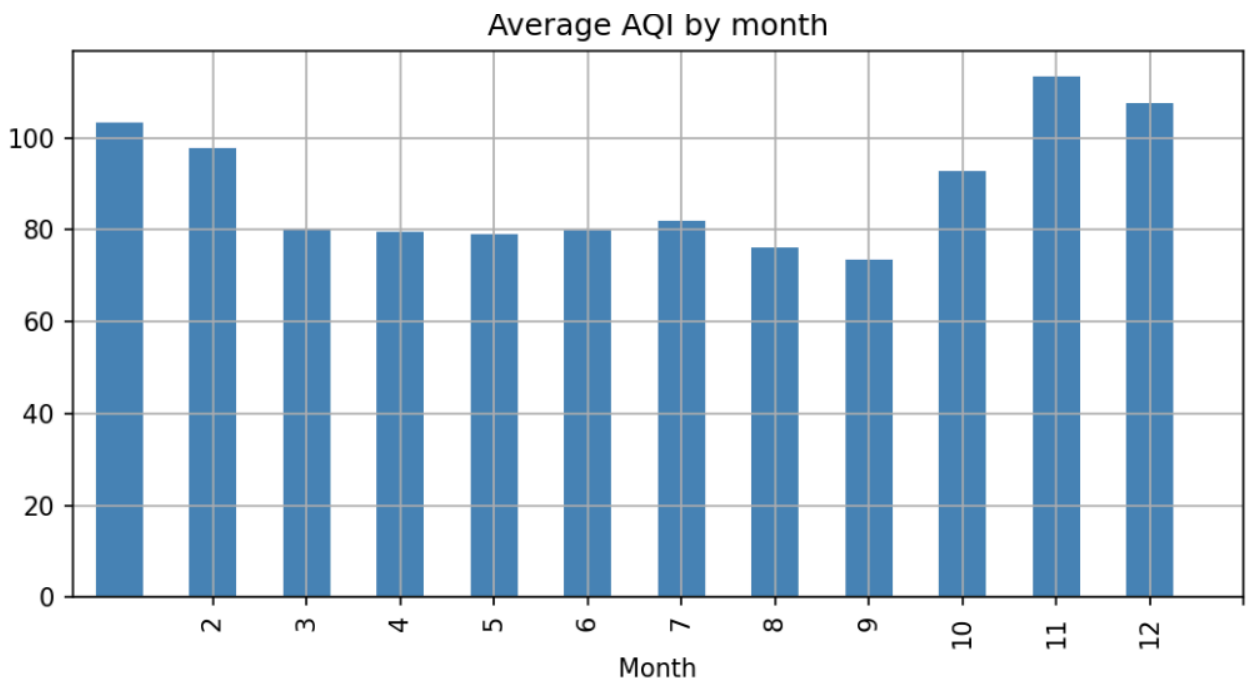
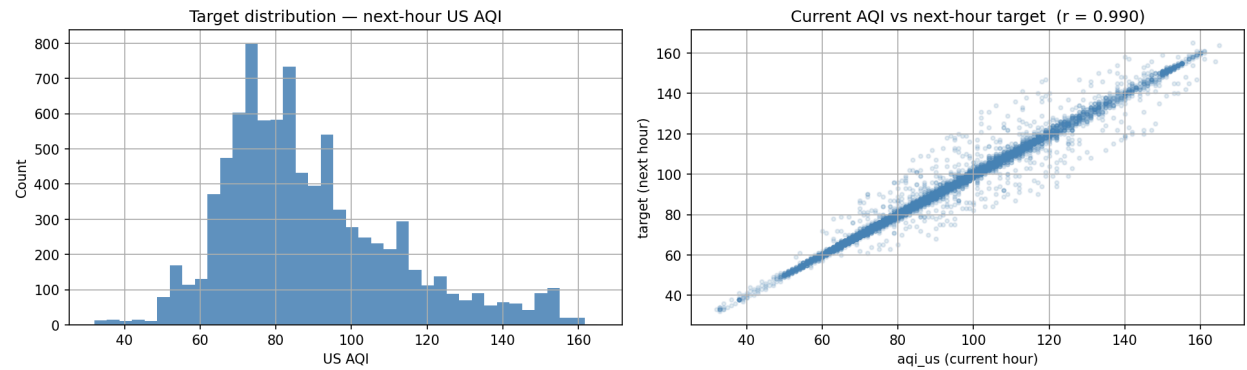
(ii): AQI peaks in early morning (06:00–09:00) and evening (18:00–22:00).



(iii): Higher wind speed correlates with lower AQI (dispersion).

Some other EDA visuals that were created:





4. Model Training and Evaluation:

The best model was found to be XGBoost. The following are the comparison metrics between the different models used:

Model	Avg RMSE	Avg MAE	Avg R²
XGBoost	7.12	4.35	0.859
Random Forest	7.67	4.61	0.837
Gradient Boosting	7.99	4.91	0.823
Ridge	7.78	4.80	0.832
SVR	12.63	9.43	0.557

5. CI/CD Automation and Pipeline Used:

Two GitHub Actions workflows keep the pipeline hands-free. The Feature Pipeline runs every hour to fetch recent data and upsert MongoDB. The Training Pipeline runs daily at 02:00 UTC to retrain and register the best model. Both use MONGODB_URI from GitHub Secrets and support manual triggers via workflow dispatch.

6. Current Limitations in the Project:

(i) The next-hour prediction is strong (R^2 approx 0.86), but iterative multi-day forecasts accumulate error over longer horizons. Open-Meteo reanalysis/forecast inputs remain a ceiling on accuracy.

(ii) Forecasts rely on Open-Meteo rather than independent ground stations.

(iii) Backfill is now 365 days to capture seasonal variation. However, the storage on MongoDB Atlas free tier remains a constraint when retaining a full year of hourly engineered rows.

7. Issues faced throughout the Project:

(i) Hopsworks integration initially worked, but due to a cluster-side issue, GitHub Actions jobs were hanging indefinitely and no data was being updated in the feature store.

(ii) Migrated the feature store to MongoDB Atlas, but the free-tier 512 MB storage limit was exhausted after about two weeks, causing feature and training pipelines to fail until old data was removed or the plan was upgraded. After clearing the cluster, data was re-backfilled with 365 days under the new feature schema.

(iii) The interface has been directly deployed on streamlit cloud (free plan) so sometimes app reboots took too much loading this was also noticeable sometimes during live inference on the interface.